Guía de Actividades **PrácticoExperimentales** Nro. 005

1. Datos Generales

Asignatura	Teoría de la programación
Estudiante	Shiguango Diaz Anderson Gonzalo
Ciclo	1 A
Unidad	**1**
Resultado de aprendizaje de la unidad	Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	**005**
Tipo	Individual
Título de la Práctica	Del diseño del algoritmo con estructuras secuenciales a la construcción del programa.
Nombre del Docente	José Oswaldo Guamán Quinche
Fecha	Jueves 29 de octubre del 2026 al Viernes 20 de octubre del 2026
Horario	Jueves, 12h30 – 13h30 Viernes, 09h30 - 11h30
Lugar	Aula física asignada al paralelo.
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Desarrollar la capacidad de transformar un problema en una solución computacional.
- Aplicar estructuras secuenciales en el diseño del algoritmo en un lenguaje de programación.
- Validar la lógica del algoritmo mediante pruebas de escritorio.

3. Materiales y reactivos:

- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: Según los contenidos de la unidad.

4. Equipos y herramientas

- Computador personal con sistema operativo Windows, Linux o macOS.
- Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).

- Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.
- Conexión a internet estable para acceder a recursos digitales y software en línea.

5. Procedimiento / Metodología

Metodología de aprendizaje: aprendizaje basado en problemas.

Inicio

- Presentación del objetivo de la práctica.
- **Contextualización del problema**: Un estudiante necesita saber qué calificación debe obtener en el tercer certamen (C3) para aprobar la asignatura con una nota final de 60/100 puntos.

Para calcular la nota final se utilizan las siguientes fórmulas:

Promedio de certámenes (NC):

$$NC = \frac{C1 + C2 + C3}{3}$$

Nota final del ciclo (NF):

$$NF = (NC \cdot 0.7) + (NL \cdot 0.3)$$

Donde:

- C1 y C2 son las notas de los dos primeros certámenes.
- C3 es la nota del tercer certamen (la que se debe calcular).
- NL es la nota de laboratorio.
- NF es la nota final de la asignatura.

El programa debe permitir ingresar las notas de C1, C2 y NL; calcular automáticamente la nota mínima necesaria en C3 para que el estudiante apruebe la asignatura.

Nota: Si el resultado es negativo, significa que ya aprueba con las notas actuales.

Desarrollo

La solución del problema se desarrolla mediante la implementación del algoritmo en un lenguaje de programación, empleando **exclusivamente estructuras secuenciales**. Para ello, seguir los siguientes pasos:

1. **Análisis del problema:**
   Datos de entrada: Notas del primer certamen, segundo certamen y laboratorio.
   Proceso: Calcular la nota mínima necesaria en c3 mediante las formulas establecidas.
   Datos de salida: Nota mínima que se debe obtener en el tercer certamen.

2. **Diseño de la solución:**

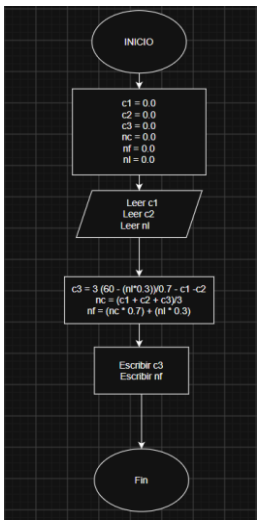


Fig [1]

3. **Codificación:**

```c
// nota.c
#include <stdio.h>

int main () {
    //varibales y constantes
    float c1,c2,c3,nl;
    const float nota_aprobacion = 60.0;
    const float peso_certamenes = 0.7;
    const float peso_laboratorio = 0.3;

    printf("Escriba la nota del primer certamen\n");
    scanf("%f", &c1);
    printf("Escriba la nota del segundo certamen\n");
    scanf("%f", &c2);
    printf("Escriba la nota de laboratorio\n");
    scanf("%f", &nl);

    c3 = 3 * ((nota_aprobacion - (nl * peso_laboratorio)) / peso_certamenes) -c1 -c2;

    printf("La nota necesaria para el tercer certamen es: %.2f \n", c3);

    return 0;


}
```

Fig [2]

4. **Pruebas de funcionamiento:**

```
C:\Users\ander\OneDrive\Documentos>nota
Escriba la nota del primer certamen
70
Escriba la nota del segundo certamen
65
Escriba la nota de laboratorio
80
La nota necesaria para el tercer certamn es: 19.29

C:\Users\ander\OneDrive\Documentos>
```

Fig [3]

c1	c2	nl	peso_laboratorio	nota_aprobacion	peso_certamenes	c3
70,00	65,00	80,00	0,30	60,00	0,70	19,29

Fig [4]

6. Preguntas de Control:

- ¿Qué es un programa con estructuras secuenciales?

  Un programa o algoritmo secuencial es aquel en el cual sus instrucciones y líneas de código se ejecutan en un orden estrictamente lineal y sucesivo, una tras otra, desde el inicio hasta llegar al final del programa. [3]

- ¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?

  Trasladar un algoritmo representado en pseudocódigo o en un diagrama de flujo hacia un lenguaje de programación formal (como Java, Python, C o C++) se conoce como la fase de codificación o transcripción. [4]

- ¿Por qué es importante validar el programa mediante casos de prueba?

  Validar un programa o algoritmo mediante casos de prueba y pruebas de escritorio es una etapa indispensable en el ciclo de desarrollo. Su importancia se sintetiza en los siguientes puntos clave:

  1. Comprobar los resultados esperados:

  Al simular o ejecutar el código con datos de entrada específicos, se sigue cada paso de las instrucciones para asegurarse de que los cálculos y los procesos generen exactamente las salidas que se esperan. Esto permite confirmar que la lógica aplicada resuelve correctamente el problema que se planteó. [4]

  2. Detectar y corregir errores semánticos y de lógica:

Mientras que el compilador solo detecta fallos en la sintaxis de las reglas del lenguaje, los errores lógicos o de funcionamiento solo se revelan al poner a prueba el programa. La validación y depuración permiten señalar con precisión la línea o instrucción donde ocurre el fallo para corregirlo a tiempo. [2]

3. Evaluar el comportamiento ante casos limite o de error:

Evaluar el comportamiento ante casos límite o errores es importante. Al someter el programa a datos difíciles o inusuales, se puede comprobar si funciona bien en situaciones extremas. Esto permite ver cómo responde ante valores fuera de rango o problemas comunes, como dividir entre cero o intentar leer un archivo que no existe. También ayuda a asegurar que se manejen correctamente las excepciones que puedan surgir. [1]

7. Bibliografía

[1] A. A. Bhuiyan and M. Amiruzzaman, Programming with Java, 2nd ed. The Pennsylvania Alliance for Design of Open Textbooks (PA-ADOPT), 2025. Disponible en: https://open.umn.edu/opentextbooks/textbooks/programmingwith-java

[2] J. E. Guerra Salazar, M. V. Ramos Valencia, and G. E. Vallejo Vallejo, Programando en C desde la práctica: problemas resueltos. Puerto Madero Editorial, 2023. Disponible en: https://dialnet.unirioja.es/servlet/libro?codigo=933288

[3] J. M. Toro Bonilla, Fundamentos de programación: Python, 2.ª ed. Editorial Universidad de Sevilla, 2023. Disponible en: https://dialnet.unirioja.es/servlet/libro?codigo=871117

[4] M.M. Arteaga Martínez, Lógica de programación con PSeInt: enfoque practico, 1.era ed. Medellín, Colombia: Corporación Universitaria Remington, dic. 2023. Doi: 10.22209/9786289585247. Disponible en: https://notebook.google.com/notebook/9ee935aa-a0a9-4473-9c54-1abe99677691?authuser=2&pageId=none#:~:text=Logica_de_Programacion.pdf